In [1]:
```
#PRN: 2501132042
#title: slr on salary
```

In [ ]:
```
import pandas as pd
```

In [8]:
```
df= pd.read_csv("Salary_dataset.csv")
```

In [9]:
```
df.head()
```

Out[9]:

	Unnamed: 0	YearsExperience	Salary
0	0	1.2	39344.0
1	1	1.4	46206.0
2	2	1.6	37732.0
3	3	2.1	43526.0
4	4	2.3	39892.0

In [10]:
```
df.columns
```

Out[10]:
```
Index(['Unnamed: 0', 'YearsExperience', 'Salary'], dtype='object')
```

In [11]:
```
df.shape
```

Out[11]:
```
(30, 3)
```

In [13]:
```
df.isnull().sum()
```

Out[13]:
```
Unnamed: 0         0
YearsExperience    0
Salary             0
dtype: int64
```

In [14]:
```
df.describe
```

Out[14]:
```
<bound method NDFrame.describe of     Unnamed: 0  YearsExperience    Salary
0            0              1.2   39344.0
1            1              1.4   46206.0
2            2              1.6   37732.0
3            3              2.1   43526.0
4            4              2.3   39892.0
5            5              3.0   56643.0
6            6              3.1   60151.0
7            7              3.3   54446.0
8            8              3.3   64446.0
9            9              3.8   57190.0
10          10              4.0   63219.0
11          11              4.1   55795.0
12          12              4.1   56958.0
13          13              4.2   57082.0
14          14              4.6   61112.0
15          15              5.0   67939.0
16          16              5.2   66030.0
17          17              5.4   83089.0
18          18              6.0   81364.0
19          19              6.1   93941.0
20          20              6.9   91739.0
21          21              7.2   98274.0
22          22              8.0  101303.0
23          23              8.3  113813.0
24          24              8.8  109432.0
25          25              9.1  105583.0
26          26              9.6  116970.0
27          27              9.7  112636.0
28          28             10.4  122392.0
29          29             10.6  121873.0>
```

In [15]:
```
df.corr()
```

Out[15]:

	Unnamed: 0	YearsExperience	Salary
Unnamed: 0	1.000000	0.986460	0.960826
YearsExperience	0.986460	1.000000	0.978242
Salary	0.960826	0.978242	1.000000

In [22]:
```
# remove unwanted column
df.drop('YearsExperience        ', axis=1,inplace=True)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[22], line 2
      1 # remove unwanted column
----> 2 df.drop(                        , axis=1,inplace=True)

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\frame.py:5603, in Dat
aFrame.drop(self, labels, axis, index, columns, level, inplace, errors)
   5455 def drop(
   5456     self,
   5457     labels: IndexLabel | None = None,
   (...)   5464     errors: IgnoreRaise = "raise",
   5465 ) -> DataFrame | None:
   5466     """
   5467     Drop specified labels from rows or columns.
   5468
   (...)   5601             weight  1.0     0.8
   5602     """
-> 5603     return super().drop(
   5604         labels=labels,
   5605         axis=axis,
   5606         index=index,
   5607         columns=columns,
   5608         level=level,
   5609         inplace=inplace,
   5610         errors=errors,
   5611     )

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\generic.py:4810, in N
DFrame.drop(self, labels, axis, index, columns, level, inplace, errors)
   4808 for axis, labels in axes.items():
   4809     if labels is not None:
-> 4810         obj = obj._drop_axis(labels, axis, level=level, errors=errors)
   4812 if inplace:
   4813     self._update_inplace(obj)

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\generic.py:4852, in N
DFrame._drop_axis(self, labels, axis, level, errors, only_slice)
   4850         new_axis = axis.drop(labels, level=level, errors=errors)
   4851     else:
-> 4852         new_axis = axis.drop(labels, errors=errors)
   4853     indexer = axis.get_indexer(new_axis)
   4855 # Case for non-unique axis
   4856 else:

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:7136,
in Index.drop(self, labels, errors)
   7134 if mask.any():
   7135     if errors != "ignore":
-> 7136         raise KeyError(f"{labels[mask].tolist()} not found in axis")
   7137     indexer = indexer[~mask]
   7138 return self.delete(indexer)

KeyError: "['YearsExperience\\t'] not found in axis"
```

In [23]:
```
# find correlation
df.corr()
```

Out[23]:

	YearsExperience	Salary
YearsExperience	1.000000	0.978242
Salary	0.978242	1.000000

In [24]:
```
#plot scatter diagram
import matplotlib.pyplot as plt
x=df['YearsExperience']
y=df['Salary']
plt.scatter(x,y)
plt.xlabel('YearsExperience')
plt.ylabel('Salary')
plt.title('Scatter plot of yeras of exp and salary')
```

Out[24]:
```
Text(0.5, 1.0, 'Scatter plot of yeras of exp and salary')
```

In [25]:
```
# to create linear model
from sklearn.model_selection import train_test_split
```

In [32]:
```
X=df[['YearsExperience']]
y=df['Salary']
```

In [33]:
```
X_train, X_test, y_train, y_test= train_test_split(X,y, random_state=26,test_siz
```

In [38]:
```
X_train.shape
```

Out[38]:
```
(24, 1)
```

In [35]:
```
X_test.shape
```

Out[35]: (6, 1)

In [36]:
```
y_train.shape
```

Out[36]: (24,)

In [37]:
```
y_test.shape
```

Out[37]: (6,)

In [51]:
```
model.score(X_train,y_train)
```

Out[51]: 0.9460054870434312

In [52]:
```
model.score(X_test,y_test)
```

Out[52]: 0.9835849730044816

In [39]:
```
# import the model and train it
from sklearn.linear_model import LinearRegression
```

In [40]:
```
model=LinearRegression()
```

In [41]:
```
model.fit(X_train, y_train)
```

Out[41]:

▾ LinearRegression

▶ Parameters

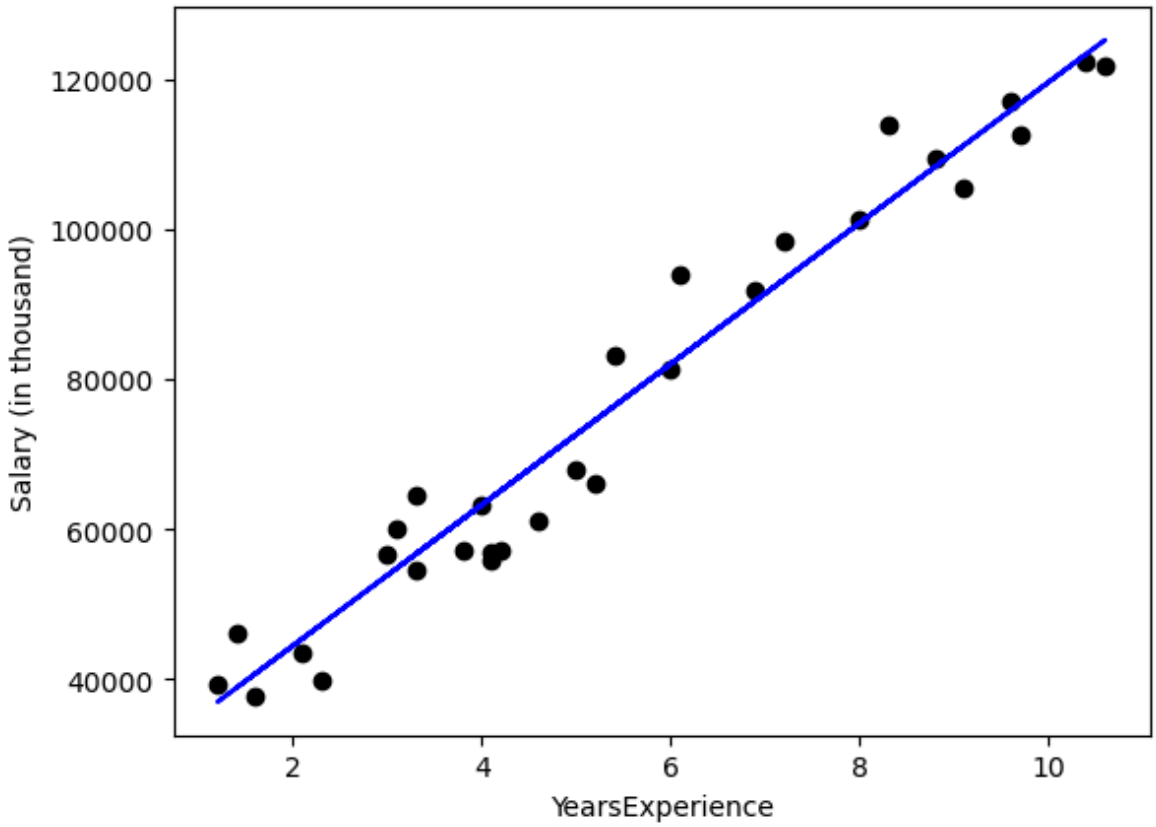
In [42]:
```
model.score(X_train, y_train)
```

Out[42]: 0.9460054870434312

In [43]:
```
y_pred=model.predict(X_train)
y_valtestpred=model.predict(X_test)
```

In [54]:
```
# to plot the best fit line
plt.scatter(X,y,color='black')
plt.plot(X_train, y_pred, color='blue')
plt.xlabel('YearsExperience')
plt.ylabel('Salary (in thousand)')
```

Out[54]: Text(0, 0.5, 'Salary (in thousand)')



In [47]:
```
model.predict(X_train)
```

Out[47]:
```
array([100807.32890847, 125231.5020826 ,  53837.76511207,  90474.02487326,
        56655.93893985,  64171.06914728, 111140.63294368,  56655.93893985,
        82019.50338991,  63231.67787135,  72625.59063063,  68868.02552692,
        76383.15573434, 103625.50273626,  40686.28724907,  47262.02618057,
        36928.72214536,  65110.4604232 ,  82958.89466584, 123352.71953075,
        38807.50469722,  74504.37318249,  54777.156388  ,  93292.19870105])
```

In [48]:
```
model.predict(X_test)
```

Out[48]:
```
array([115837.58932332,  45383.24362871, 116776.98059925,  64171.06914728,
       108322.4591159 ,  61352.89531949])
```

In [55]:
```
# create a data frame of actual and predicted salary
y_test
```

Out[55]:
```
26    116970.0
3      43526.0
27    112636.0
12     56958.0
24    109432.0
9      57190.0
Name: Salary, dtype: float64
```

In [56]:
```
y_valtestpred
```

Out[56]:
```
array([115837.58932332,  45383.24362871, 116776.98059925,  64171.06914728,
       108322.4591159 ,  61352.89531949])
```

In [61]:
```
pd.DataFrame({'Actual Salary':y_test, 'predicted salary':y_valtestpred})
```

Out[61]:

	Actual Salary	predicted salary
26	116970.0	115837.589323
3	43526.0	45383.243629
27	112636.0	116776.980599
12	56958.0	64171.069147
24	109432.0	108322.459116
9	57190.0	61352.895319

In [62]:
```
DF1=pd.DataFrame({'Actual Salary':y_test, 'predicted salary':y_valtestpred})
```

In [65]:
```
# to send the data to boss
df.to_csv('DF1')
```

model evaluation

In [67]:
```
# to find slope and intercept
print('slope is', model.coef_)
print('intercept is', model.intercept_)
```

```
slope is [9393.91275928]
intercept is 25656.026834224947
```

In [69]:
```
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
```

In [70]:
```
mean_absolute_error(y_test,y_valtestpred)
```

Out[70]: 3269.356709252484

In [71]:
```
r2_score(y_test,y_valtestpred)
```

Out[71]: 0.9835849730044816

In [72]:
```
import numpy as np
```

In [73]:
```
np.sqrt(mean_squared_error(y_test,y_valtestpred))
```

Out[73]: np.float64(3925.7392742927)

In [ ]: